Team Cacttus Shield

Cacttus Education

(Team 16)

White Team Documentation

Table of Contents

Public Services

Network Architecture

Servers

Pre-Remediation Red Team Attack Proof-of-Concepts

Wazuh implementation

Backup Solution

White Team Documentation

Public Services(Setup for Service Scanner)

We have the following services publically available:

Domain Name	IP Address	Port
Ad.team16.isucdc.com	168.84.5.10	383, 636, 3389 (LDAP, LDAPS and RDP)
Tickets.team16.isucdc.com	168.84.5.20	3389, 5000 (RDP and HTTP)
Mgmt.team16.isucdc.com	168.84.5.40	22, 5000 (SSH and HTTP)
Admin.team16.isucdc.com	168.84.5.40	80, 3389 (HTTP and RDP)
www.team16.isucdc.com	168.84.5.50	22, 80 (SSH and HTTP)

Introduction and Purpose of each Service

Domain Controller (ad.team16.isucdc.com) Manages user authentication and logins across all machines and websites in the environment. Provides certificates used by the ticketing machine.

Ticketing Machine (tickets.team16.isucdc.com) Runs the application that issues and validates tickets for buses and trains. Simulates the city's backend ticketing system and physical ticket readers.

Management Backend (mgmt.team16.isucdc.com) Serves as the main transit management system. Handles user authentication, ticket payments, fleet management, route management, and communication with buses/trains.

Admin User Interface (admin.team16. isucdc.com) Hosts the internal admin portal. Used by transit staff to view transportation system status and manage the fleet of vehicles.

Public Website (www.team16.isucdc.com) Hosts the public-facing citizen website. Allows users to sign up, view routes, manage accounts, buy tickets, and scan/validate tickets.

Additional Appliances (For Internal Usage and Security Purposes)

Sophos Firewall (168.84.5.100) Acts as the main network perimeter defense. Controls and inspects all traffic entering and leaving the team network, routes traffic between internal hosts, enforces security policies, and blocks threats.

Wazuh Server (168.84.5.180) Central security monitoring and threat detection platform. Collects logs from all machines, performs intrusion detection, file integrity monitoring, vulnerability scanning, and generates real-time security alerts.

Backup Server (168.84.5.250) Manages automated backups of critical data and systems across the environment. Enables quick recovery and restoration of machines after failures.

Network Architecture

e implemented a secure, segmented network architecture by placing **all** scenario resources (Active Directory, Ticketing Machine, Management Backend, Admin UI, Public Website, Wazuh Server, and Backup Server) behind a dedicated **Sophos SFOS 25.5.1 Firewall**.

The Sophos appliance now serves as the single point of entry and exit for all traffic. It actively routes **two networks**:

- **Public-facing / External Network (172.20.10.0/24 – 168.84.5.0/24)** (connected to the competition-wide network) used for inbound access to publicly required services and WAN access for hosts.

- **Internal Production Network (10.10.10.0/24)** – the primary LAN where all hosts reside.

To make specific services reachable from the competition network (e.g., HTTP/80 on Public Website and Admin UI, HTTP/5000 on Ticketing and Management Backend, RDP/3389, SSH/22, LDAP/389, LDAPS/636), the firewall performs **Static Destination NAT (DNAT)** combined with port-forwarding rules. This maps external competition-network IP addresses and ports to the corresponding internal IPs and ports of each host. Because we use static (one-to-one or port-specific) NAT,

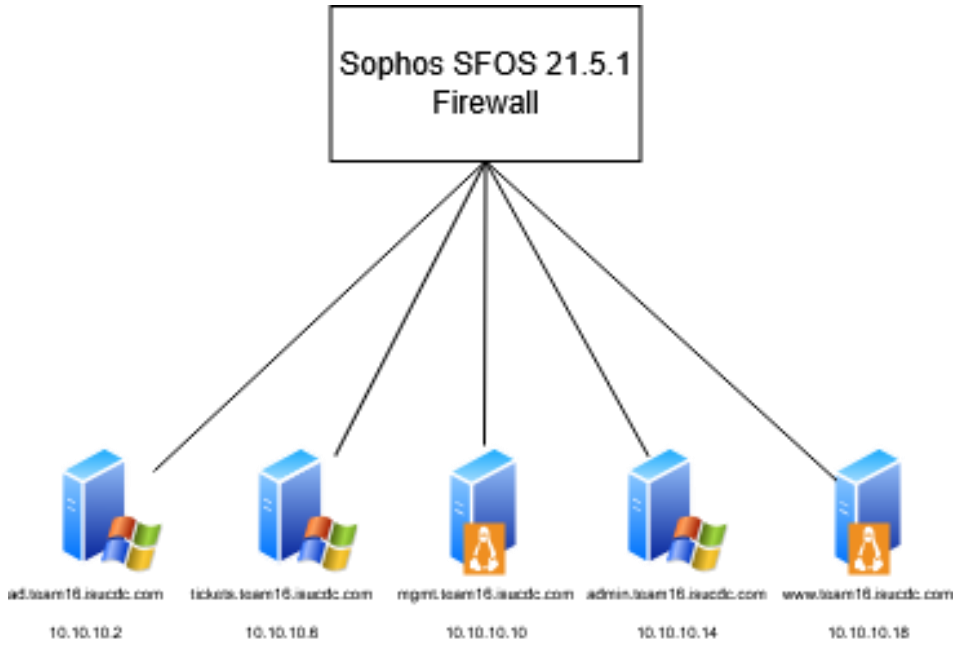
The mappings are persistent and predictable, which is essential for reliable service availability and for passing competition checks.
In parallel, we created a **dedicated secondary internal network** exclusively for host-to-host and application-to-application communication within our team. Examples include:

1. Active Directory authenticating all domain-joined machines
2. Frontend Server Communicating with Backend Server

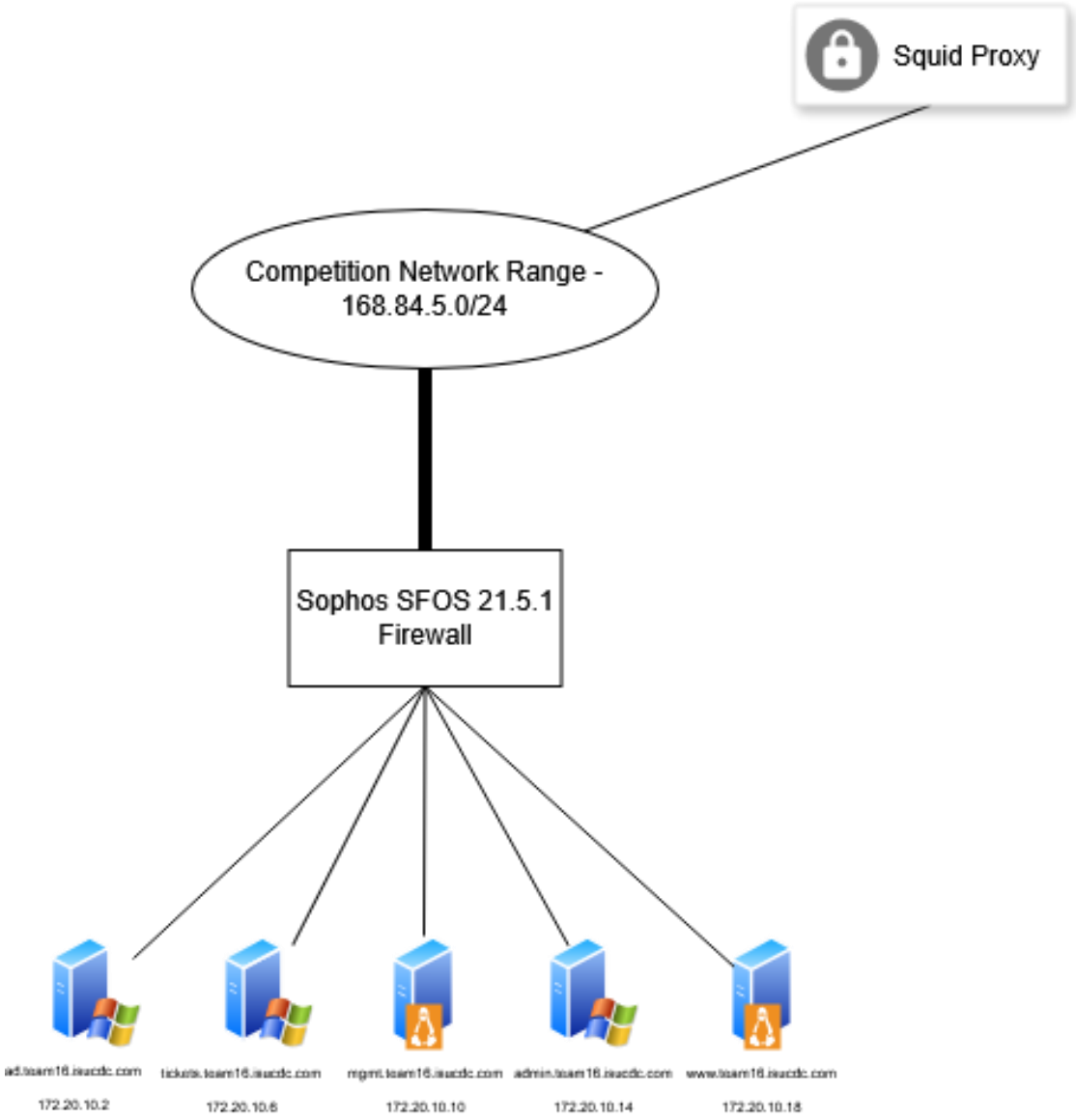
This second internal network is **not** directly accessible from the competition network — all traffic between the two internal networks and the external network is forced to pass through the Sophos firewall.

Key Security and Design Benefits

- **No traffic bypass possible**: Every packet leaving or entering any host must pass firewall inspection, policy enforcement, and threat scanning. There are no “backdoor” routes or direct Layer-2 connections to the competition network.
- **Strong network segmentation & isolation**: Public services are isolated from internal management traffic. Internal-only services (e.g., AD LDAP for machine



Internal Network IP Addressing Diagram



External Network IP Addressing and Mapping Diagram

Each host in our environment is connected directly to the Sophos SFOS 25.5.1 Firewall using a dedicated point-to-point (/30) link. To implement this, we divided the 10.10.10.0/24 address range into five separate /30 subnets and the 172.20.10.0/24 range into five additional /30 subnets. Using a 255.255.255.252 subnet mask, each /30 provides exactly two usable IP addresses, one assigned to the firewall and one to the host.

This configuration creates a strict star topology with the firewall at the center of all network communications. Because no two hosts share the same subnet or Layer-2 broadcast domain, they remain completely isolated from one another at the data link layer. As a result, broadcast traffic, multicast packets, and ARP requests cannot travel directly between hosts, guaranteeing that no traffic can bypass the firewall's restrictions.

All inter-host, inbound, and outbound traffic is forced to pass through Layer-3 routing on the firewall, where we apply comprehensive stateful packet inspection. Filtering decisions are made based on specific ports, traffic direction, and connection states such as NEW, ESTABLISHED, and RELATED. This ensures tight control, with only explicitly permitted flows allowed through the firewall.

Inbound IDS/IPS on DMZ Network

All inbound traffic originating from the competition network enters our environment through the Sophos SFOS 25.5.1 Firewall's DMZ interfaces, where a full-featured Intrusion Detection and Prevention System (IDS/IPS) is enabled. This security layer performs inspection and real-time analysis to identify and automatically block known attack signatures, exploit attempts, malware payloads, command-and-control communications, and anomalous behavior before any traffic can reach our publicly exposed services such as the Public Website, Admin User Interface, or Ticketing Machine endpoints.

Outbound SSL Inspection for Selected Hosts and Destinations

For carefully selected internal hosts that require outbound connectivity, we have activated full SSL/TLS inspection on the Sophos firewall. This capability decrypts HTTPS traffic leaving those hosts, performs thorough content inspection for hidden threats such as data exfiltration, malware callbacks, command-and-control channels, and policy violations, then re-encrypts and forwards the inspected traffic, ensuring that encrypted outbound sessions cannot be used to bypass our security controls.

Web Application Firewall on Inbound Web Traffic

All inbound HTTP and HTTPS traffic destined for our web-based applications is routed through an integrated Web Application Firewall (WAF) on the Sophos firewall. This specialized layer-7 protection engine analyzes every web request in detail to detect and block OWASP Top 10 threats including SQL injection, cross-site scripting (XSS), , and other application-layer attacks, thereby safeguarding the Public Website, Admin User Interface, and Management Backend from sophisticated web-based exploits while maintaining full service availability.

Firewall Licensing and Signature Update Challenges

We encountered notable licensing activation and threat signature gathering issues with the Sophos SFOS 25.5.1 Firewall throughout the deployment and operation of our environment. These difficulties arose directly from the competition-provided upstream proxy, which lacked full support for DNS resolution and reliable outbound communication with several critical Sophos domains required for license validation, renewal, and the automated download of the latest security intelligence updates. As a result, the firewall was unable to consistently maintain an active licensed state or retrieve fresh IPS, antivirus, web application firewall, and application control signatures in real time.

This proxy-related limitation not only restricted the availability and effectiveness of several security features but also increased the overall load on the appliance's hardware resources. The repeated failed update attempts and fallback to outdated local databases forced the firewall to operate with reduced optimization, higher CPU and memory utilization during inspection tasks.

Restriction of Legacy Name Resolution and Remote Management Protocols for Internal Traffic

To further minimize the internal attack surface and prevent common lateral movement techniques, we explicitly locked down several high-risk legacy protocols for all east-west traffic between hosts. Using granular stateful firewall rules on each point-to-point internal interface of the Sophos SFOS 25.5.1 appliance, we blocked LLMNR (UDP port 5355), NBTNS/NetBIOS Name Service (UDP/TCP port 137), mDNS/Multicast DNS (UDP port 5353), and WinRM/Windows Remote Management (TCP ports 5985 and 5986) for communication between any of our hosts.

These restrictions eliminate well-known attack vectors frequently exploited in Windows environments, including LLMNR and NBT-NS poisoning for credential theft, multicast-based discovery abuse, and unauthorized remote execution or management access via WinRM. Because every internal packet must traverse the firewall, these blocks are enforced universally and cannot be bypassed at Layer 2 or Layer 3, providing consistent defense across the entire segmented environment even if an individual host is compromised.

This hardening measure complements our point-to-point topology and stateful filtering, significantly raising the difficulty for Red Team operations while maintaining full required functionality for legitimate services.

Servers

Overall

Domain Controller Hardening The Active Directory Domain Controller was hardened by first removing all unnecessary local user accounts to strictly enforce the principle of least privilege. We disabled the Remote Registry service entirely, applied every available Windows security update and patch, and used the Sophos firewall to restrict inbound access to only the essential ports required for domain operations. Cynet XDR was installed to deliver advanced endpoint detection and response capabilities, while the Wazuh agent was configured to forward comprehensive system logs and security events for centralized visibility and correlation.

Tickets Server Hardening On the Ticketing Machine, we eliminated unnecessary user accounts and completely disabled the Remote Registry service to shrink the attack surface. All current Windows security patches were installed, and non-essential ports were locked down at the firewall level to allow only required ticketing and certificate traffic. Sysmon was deployed with a custom configuration to capture detailed process creation, network connections, and file access events, with logs streamed to our central Wazuh server. Cynet XDR was also installed to provide real-time behavioral analysis and threat prevention.

Management Server Hardening The Management Backend received thorough Linux-centric hardening. We reviewed and tightened sudo permissions and user rights, adjusted ownership on critical directories and files, and applied the immutable attribute (chattr +i) to sensitive configuration files to prevent modification even by root. All packages were upgraded to their latest versions, SELinux was enabled in enforcing mode, and the system underwent a full security audit with Lynis. Sysdig was installed for deep kernel-level visibility, Cynet XDR provided endpoint protection, and the Wazuh agent ensured all events were centralized for monitoring.

Admin Server Hardening For the Admin User Interface server, we removed all unnecessary local user accounts and disabled the Remote Registry service to minimize exposure. The latest Windows security updates and patches were fully applied, and access to non-required ports was strictly restricted through the firewall. Sysmon was configured for detailed logging of system activity and process execution, complemented by Cynet XDR for advanced threat detection and response. All logs from both tools are forwarded to our central Wazuh platform for real-time correlation and alerting.

WWW Server Hardening The Public Website server was hardened following Linux best practices. We tightened user permissions and sudo rights, updated ownership on all sensitive application directories and files, and applied the immutable flag (chattr +i) to key configuration and binary files to protect them against tampering. All system packages were upgraded to the newest versions, SELinux was activated in enforcing mode, and the entire system was audited using Lynis for compliance and vulnerability assessment. Cynet XDR was deployed for endpoint protection, with complete visibility achieved through the Wazuh agent forwarding logs to our central security platform.

Wazuh Server Network Hardening

We secured the central Wazuh Server by implementing precise firewall rules that restrict agent communication ports 1514 (UDP for event forwarding), 1515 (TCP for agent enrollment), and 5500 (TCP for additional Wazuh services) to accept traffic exclusively from the specific internal IP addresses assigned to our monitored hosts. This ensures that only authorized agents within our point-to-point segmented environment can register and send logs, while blocking all other sources at the Sophos firewall layer. Port 443 (HTTPS) was selectively opened to allow secure access to the Wazuh dashboard and API, enabling centralized management, real-time alerting, threat hunting, and security event analysis from authorized internal or competition-network management stations. In contrast, SSH access on port 22 was fully restricted across all interfaces, completely eliminating remote administrative login to this high-value security platform and further reducing its attack surface.

Active Directory Hardening

We performed an exceptionally thorough, defense-in-depth hardening of the Active Directory Domain Controller (**ad.team16.isucdc.com**) running Windows Server 2016. As the central identity, authentication, domain-join, Kerberos ticket-granting, Group Policy distribution, and Certificate Authority server for the entire ICDC 2026 environment, this system was treated as our highest-value asset. Any compromise would have immediately violated multiple competition requirements: every scenario machine must remain domain-joined, the Domain Controller must stay reachable from the competition network on LDAP (389) and LDAPS (636), the Ticketing Machine must be able to request certificates, and IT Administrators must retain full RDP (3389) access with Administrator-level privileges. Our hardening followed an “assume breach” mindset while guaranteeing 100 % service availability. We focused on strict identity segmentation, complete cryptographic remediation of ADCS vulnerabilities, enterprise-wide policy enforcement, aggressive application whitelisting, network-layer restrictions via the Sophos firewall, and continuous visibility through Cynet XDR and Wazuh. The resulting configuration successfully neutralized Red Team attempts at credential dumping, lateral movement, ADCS abuse, Kerberoasting, LLMNR/NBT-NS poisoning, NTLM relay, and living-off-the-land execution.

Security Group Redesign and Strict Role-Based Access Control (RBAC)

The foundation of our Active Directory hardening was a complete redesign of the security group structure to enforce least privilege and clear separation of duties. In a default Windows Server 2016 domain, excessive group memberships often create privilege-escalation and lateral-movement paths. We eliminated these risks by creating four new domain global security groups and migrating every account accordingly:

- **TEAM16\Citizens** – All public citizen accounts were placed exclusively in this group. Members have no logon rights to any server, no membership in any privileged groups, and are explicitly denied access to all internal file shares, RDP sessions, and services. Their only permitted interaction is with the Public Website (www.team16.isucdc.com) over HTTP port 80 for ticket purchases and route viewing.
- **TEAM16\TransitAdmins** – Granted the exact minimum rights needed to manage the Management Backend (mgmt.team16.isucdc.com) and Admin User Interface (admin.team16.isucdc.com). Members can log on only to those two servers, have no Domain Admin rights, no local admin rights on the Domain Controller, Ticketing Machine, or Public Website server, and cannot request certificates or access sensitive paths such as C:\Windows\System32 on any host.

- **TEAM16\TicketMaintenance** – Restricted solely to the Ticketing Machine (tickets.team16.isucdc.com). Members can maintain the ticketing application, request certificates from the CA, and perform operational tasks but have zero access to any other system in the domain.
- **TEAM16\IT Administrators** – The only group with full Domain Admins membership and local Administrator rights on every domain-joined machine. This satisfies the competition requirement for full administrative access from the competition network while remaining subject to our AppLocker policy and port restrictions.

We then conducted a full audit and cleanup of all existing memberships. Unnecessary accounts were removed from Domain Admins, Enterprise Admins, Schema Admins, and the local Administrators group on every server. Delegated permissions on Organizational Units and the Certificate Authority were reviewed and corrected to prevent over-privileging. This RBAC model ensures that even if a single account is compromised, the blast radius is tightly contained to one role and a limited set of systems.

Enterprise Admins	Security Group - Universal
Enterprise Key Admins	Security Group - Universal
Enterprise Read-only Domain Controllers	Security Group - Universal
Schema Admins	Security Group - Universal
Citizens	Security Group - Global
Cloneable Domain Controllers	Security Group - Global
DnsUpdateProxy	Security Group - Global
Domain Admins	Security Group - Global
Domain Computers	Security Group - Global
Domain Controllers	Security Group - Global
Domain Guests	Security Group - Global
Domain Users	Security Group - Global
Group Policy Creator Owners	Security Group - Global
IT Administrators	Security Group - Global
Key Admins	Security Group - Global
Protected Users	Security Group - Global
Read-only Domain Controllers	Security Group - Global
Ticket Maintenance	Security Group - Global
Transit Admins	Security Group - Global
Allowed RODC Password Replication Group	Security Group - Domain Local
Cert Publishers	Security Group - Domain Local
Denied RODC Password Replication Group	Security Group - Domain Local
DnsAdmins	Security Group - Domain Local
RAS and IAS Servers	Security Group - Domain Local

Screenshot From "Active Directory Users and Groups", highlighting newly created user group

Active Directory Certificate Services (ADCS) Remediation – Eliminating ESC7 & ESC8 Vulnerabilities

The original environment contained two critical ADCS template misconfigurations known as ESC7 and ESC8. These allowed any authenticated domain user to request certificates with arbitrary Subject Alternative Names (SANs) or overly permissive issuance policies, enabling attackers to impersonate any user or machine and achieve domain dominance using tools such as Certipy. We fully remediated these vulnerabilities through a structured process:

- Identified the vulnerable templates using Certipy and the Certificate Authority MMC snap-in.
- Disabled the original templates and created new hardened versions with strict controls.
- Removed “Enroll on behalf of another user” and “Supply in the request” options.
- Restricted enrollment permissions exclusively to the legitimate security groups (e.g., TEAM16\ADS, TEAM16\TICKETS, TEAM16\ITAdmins).
- Revoked all previously issued vulnerable certificates and forced re-issuance through the new templates.

All certificate requests are now tightly audited and controlled. The sanitized request queue below shows only legitimate requests from TEAM16\ADS, TEAM16\TICKETS, and other authorized accounts after remediation. This completely eliminated the ESC7 and ESC8 attack paths while still allowing the Ticketing Machine to request the certificates required for ticket validation.

Request ID	Requester Name	Binary Certificate	Certificate Template
2	TEAM16\AD$	-----BEGIN CERTI...	Domain Controller (...
3	TEAM16\TICKETS$	-----BEGIN CERTI...	TicketSigning (Ticket...
5	TEAM16\krystal.gray	-----BEGIN CERTI...	LDAPSMisconfigure...
6	TEAM16\p4pq	-----BEGIN CERTI...	LDAPSMisconfigure...

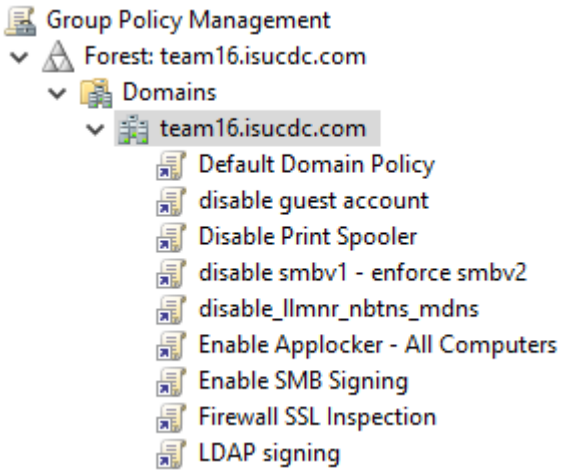
the Certificate Requests console image showing Request ID, Requester Name

Group Policy Objects – Domain-Wide Security Baseline

We created and linked a single enforced Group Policy Object at the domain root level named “team16.isucdc.com – Security Baseline”. This GPO applies uniformly to all computers and users with no exceptions and contains dozens of critical security settings that directly address common attack vectors. The full list of enabled policies is visible in the Group Policy Management console screenshot below and includes:

- Disable Guest Account – completely removed the built-in guest account.
- Disable Print Spooler – eliminated PrintNightmare-style attacks.
- Disable SMBv1 / Enforce SMBv2 – removed legacy protocol vulnerabilities.
- Disable LLMNR, NBT-NS, mDNS – blocked name-resolution poisoning (a technique Red Team attempted multiple times).
- Enable SMB Signing on all computers – prevented NTLM relay attacks.
- Enforce LDAP Signing – prevented clear-text LDAP binds and ensured LDAPS-only communication.
- Distributed Firewall SSL Certificate
- Additional settings for strong password policy, account lockout thresholds, Kerberos hardening, PowerShell logging, and disabling of unnecessary services.

Because the GPO is enforced at the domain level, the settings apply even to newly joined machines and cannot be overridden locally. This uniform enforcement complements our point-to-point firewall topology and ensures consistent security posture across the entire environment.



Screenshot Showing all Applied GPO’s

AppLocker –Restrictive Application Whitelisting

One of our most powerful controls was the deployment of a domain-wide AppLocker policy that functions as a strict allow-list with aggressive deny rules. AppLocker evaluates execution requests based on publisher, path, or file hash and operates at the kernel level, making it extremely difficult to bypass. We configured separate rule collections for Executable files, Scripts, Windows Installer packages, and DLLs with a default-deny stance.

The deny rules are comprehensive and target the majority of living-off-the-land binaries (LOLBins) and common Red Team tools:

- Deny PowerShell.exe, PowerShell ISE, cmd.exe, wscript.exe, cscript.exe, mshta.exe, regsvr32.exe, rundll32.exe, msbuild.exe, and dozens of other Microsoft binaries for all users.
- Deny all unsigned scripts and any script executing from user-writable directories (%TEMP%, %APPDATA%, etc.).
- Deny .NET host executables, Conhost, and common post-exploitation frameworks.
- Deny execution of any binary from temporary or user-controlled paths.

Allow rules were kept extremely narrow and specific:

- Allow only Microsoft-signed executables and DLLs via publisher rules.
- Allow our own code-signed backup and maintenance scripts via file-hash rules.
- Provide a temporary path-based exception for TEAM16\IT Administrators to run cmd.exe (tagged so it can be revoked instantly if needed).
- Allow required runtime components for the web applications (e.g., Mozilla VCRUNTIME via file hash).

Action	User	Name	Condition
Deny	Everyone	__MS Recommended Block: ADDINUTI...	Publisher
Deny	Everyone	__MS Recommended Block: FSIANYCP...	Publisher
Deny	Everyone	__MS Recommended Block: NTSD.EXE	Publisher
Deny	Everyone	Block PWSH.exe outside Normal Direct...	Publisher
Deny	Everyone	__MS Recommended Block: TEXTTRA...	Publisher
Deny	Everyone	---- Starter Rules: Block Powershell ISE ...	Publisher
Deny	Everyone	__MS Recommended Block: VISUALUI...	Publisher
Deny	Everyone	__MS Recommended Block: ASPNET_...	Publisher
Allow	Everyone	---- Starter Rules: Allow All Signed Exec...	Publisher
Deny	Everyone	---- Starter Rules: Block Powershell Out...	Publisher
Deny	Everyone	__MS Recommended Block: FSI.EXE	Publisher
Deny	Everyone	---- Starter Rules: Block PWSH Outside ...	Publisher
Deny	Everyone	__MS Recommended Block: MSBUILD....	Publisher
Deny	Everyone	__MS Recommended Block: CSI.EXE	Publisher
Deny	TEAM16\Tic...	Deny Powershell.exe For Ticket Mainten...	Publisher

Screenshot Showing Part of Applocker's "Executable Rules"

This configuration renders almost every common post-exploitation technique ineffective. Even if an attacker obtains a shell, they cannot launch PowerShell, download tools, or execute most LOLBins without first bypassing AppLocker – which is extremely difficult under our layered setup. The screenshots below show the rule sets in detail.

Action	User	Name	Condition	Exceptions
Deny	Everyone	__MS Recommended Block: MICROSO...	Publisher	
Deny	Everyone	__MS Recommended Block: MICROSO...	Publisher	
Deny	Everyone	__MS Recommended Block: MSBUILD....	Publisher	
Deny	Everyone	__MS Recommended Block: MICROSO...	Publisher	
Allow	Everyone	---- Starter Rules: Allow All Signed DLLs	Publisher	
Deny	Everyone	__MS Recommended Block: SYSTEM....	Publisher	
Deny	Everyone	[TOGGLE TEMPORARY] Allow Everythin...	Path	Yes
Deny	Everyone	__MS Recommended Block: davsvc.dll	Path	
Deny	Everyone	__MS Recommended Block: webclnt.dll	Path	
Deny	Everyone	__MS Recommended Block: lxssmanag...	Path	
Allow	Everyone	Allow_Mozilla_VCRUNTIME	File Hash	

Screenshot Showing Part of Applocker's "DLL Rules"

Action	User	Name	Condition	Exceptions
Allow	Everyone	---- Starter Rules: Allow All Signed Scripts	Publisher	
Deny	Everyone	---- Starter Rules: Block init.ps1	Path	
Deny	Everyone	[TOGGLE TEMPORARY] Allow Everythin...	Path	Yes
Allow	Everyone	Allow Backup Scripts	File Hash	

Screenshot Showing Part of Applocker's "Script Rules"

Integration with Firewall, Endpoint Protection, and Monitoring

All of the above controls are reinforced by our Sophos SFOS 25.5.1 firewall's point-to-point /30 links and stateful filtering, which restrict ports such as WinRM (5985/5986), Remote Registry, LLMNR (5355), NBT-NS (137), and mDNS (5353) at the network layer. The Domain Controller also runs Cynet XDR for behavioral analysis and the Wazuh agent for centralized log forwarding, giving us real-time visibility into any attempted policy violations or anomalous activity.

Pre-Remediation Red Team Attack Proof-of-Concepts

Before applying any hardening to **ad.team16.isucdc.com**, we ran controlled Red Team simulations using only a standard domain user account from the competition network. These Proof-of-Concept (POC) attacks demonstrated the default vulnerabilities that existed in the unpatched Windows server domain.

LLMNR Poisoning Responder successfully poisoned LLMNR requests for “wpad”, sending dozens of fake responses to both IPv4 and IPv6 addresses. This would have leaked NTLMv2 hashes from any account performing name resolution.

```
[!] Error starting TCP server on port 53, check permissions or other servers running.
[*] [MDNS] Poisoned answer sent to [redacted]    for name admin.local
[*] [MDNS] Poisoned answer sent to [redacted] for name admin.local
[*] [LLMNR]  Poisoned answer sent to [redacted]  for name admin
[*] [LLMNR]  Poisoned answer sent to [redacted]  for name admin
```

Screenshot from “Responder”, demonstrating poisoning using MDNS and LLMNR

SMB Enumeration with NetExec (nxc) NetExec scanned 256 targets and fully mapped every host, OS version, and role (AD, TICKETS, ADMIN) in team16.isucdc.com. Enumeration succeeded easily before AppLocker and SMB signing were enforced.

```
┌──(bpbd㉿DESKTOP-VD1T1QW)-[~]
└─$ nxc smb 10.8.4.0/24 --shares -u 'waffles.loafer' -p '121' -d team16.isucdc.com
SMB         10.8.4.01       445    TICKETS          [*] Windows 10 \ Server SotA Build 17763 x64 (name:TICKETS) (domain:team16.isucdc.com) (signing:True) (SMBv1:None)
SMB         10.8.4.02       445    ADMIN            [*] Windows Server 2022 Build 20348 x64 (name:ADMIN) (domain:team16.isucdc.com) (signing:True) (SMBv1:None)
SMB         10.8.4.50       445    TICKETS          [-] team16.isucdc.com\waffles.loafer:121 STATUS_LOGON_FAILURE
SMB         10.8.4.01       445    AD               [-] team16.isucdc.com\waffles.loafer:121 STATUS_LOGON_FAILURE
SMB         10.8.4.02       445    ADMIN            [-] team16.isucdc.com\waffles.loafer:121 STATUS_LOGON_FAILURE
```

ADCS ESC7/ESC8 Abuse with Certipy Certipy obtained a TGT and extracted the Administrator NT hash (aad3b435b51404ee...ea4b77bf3cce1018e...) using the vulnerable default certificate templates.

```
(venv) root@wazuntu-Standard-PC-i440FX-PIIX-1996:/home/wazuntu/custom_tools# certipy auth -pfx administrator.pfx -d
c-ip 168.84.5.10
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: administrator@team16.isucdc.com
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'administrator.ccache'
[*] Trying to retrieve NT hash for 'administrator'
[*] Got hash for 'administrator@team16.isucdc.com': aad3b435b51404eeaad3b435b51404ee:ea4b77bf3cce1018e13952509e64a0
d5
(venv) root@wazuntu-Standard-PC-i440FX-PIIX-1996:/home/wazuntu/custom_tools#
```

Screenshot Showing Pre-remediation Certipy ESC7/ESC8 hash extraction

NTLMv2 Hash Capture Combined LLMNR poisoning with SMB forced a full NTLMv2-SSP hash for roger.cox@TEAM16, ready for cracking or relay.

```
[SMB] NTLMv2-SSP Client   : fe80::52c3:1853:7eae:bff6
[SMB] NTLMv2-SSP Username : TEAM16\roger.cox
[SMB] NTLMv2-SSP Hash     : roger.cox::TEAM16:cffe8945e9f00e23:CF26004C00F05CCFBECE06D3DFA5D435:0101000000000000EC
96621798DC010C314873B01DC1E600000000020008004300470041004900010 01E00570049004E002D0030004D003400470032004100450034003700550057002E00430047004100490 02E004C004F004300
41004C000300140043004700410049002E004C004F00430041004C000500140043004700410049002E004C004F00430041004C000700080000EC
96621798DC0106000400020000000800300030000000000000000100000000200000F6BF1BA4EFF53CA050F1962AC78B7AFE6F295C800B08A9BE
49C05D0EC7674CAC0A001000000000000000000000000000000000000000900100063006900660073002F003100320033000000000000000000
```

Screenshot Showing Pre-hardening NTLMv2 hash capture

These POCs proved the original domain could be fully compromised in minutes. After our GPOs, AppLocker, certificate remediation, and firewall rules, every identical attack was blocked.

Wazuh implementation

We set up a Wazuh SIEM server named Wazuh-SIEM-01 to serve as our central security monitoring and threat detection platform for the entire team16.isucdc.com environment. We chose a recent stable version of Wazuh that worked well for our needs.

The server was installed on its own dedicated machine using the standard installation process. We followed basic security practices to keep it safe, limited the running services to only what was necessary, and applied our own simple configuration so it would monitor our systems effectively.

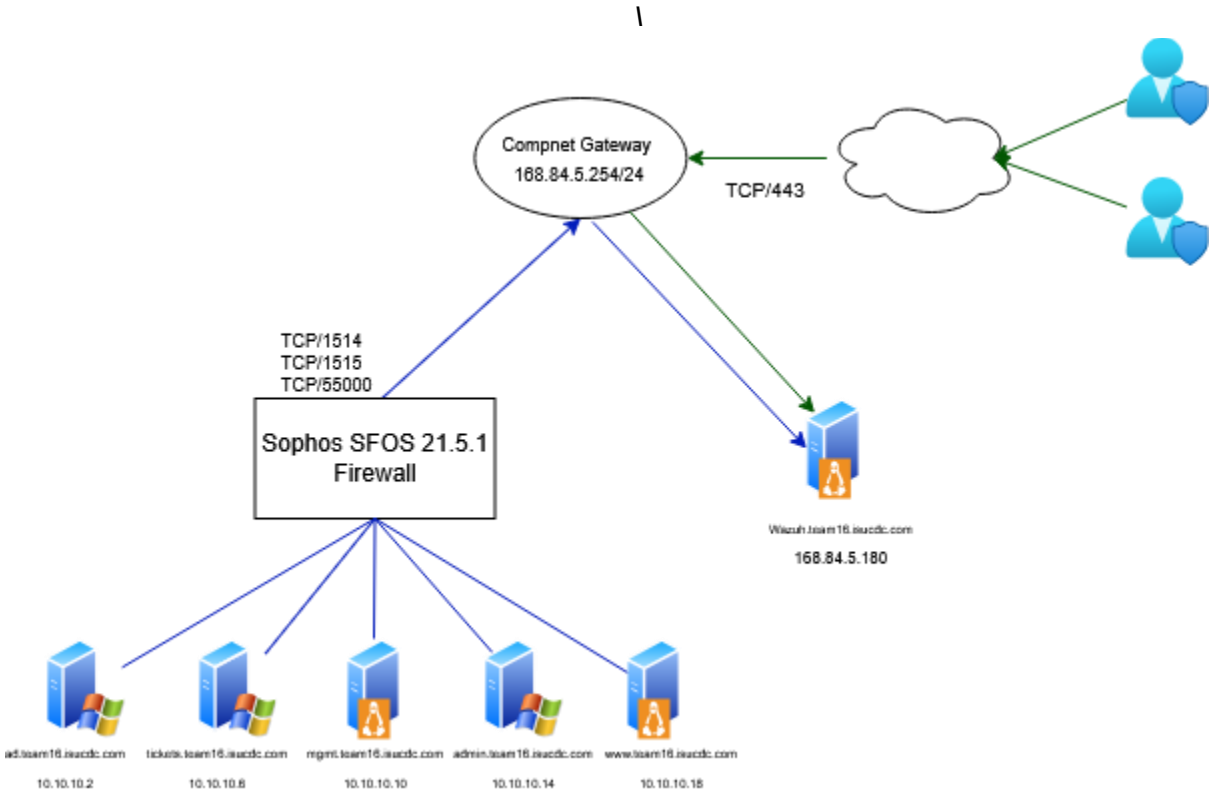
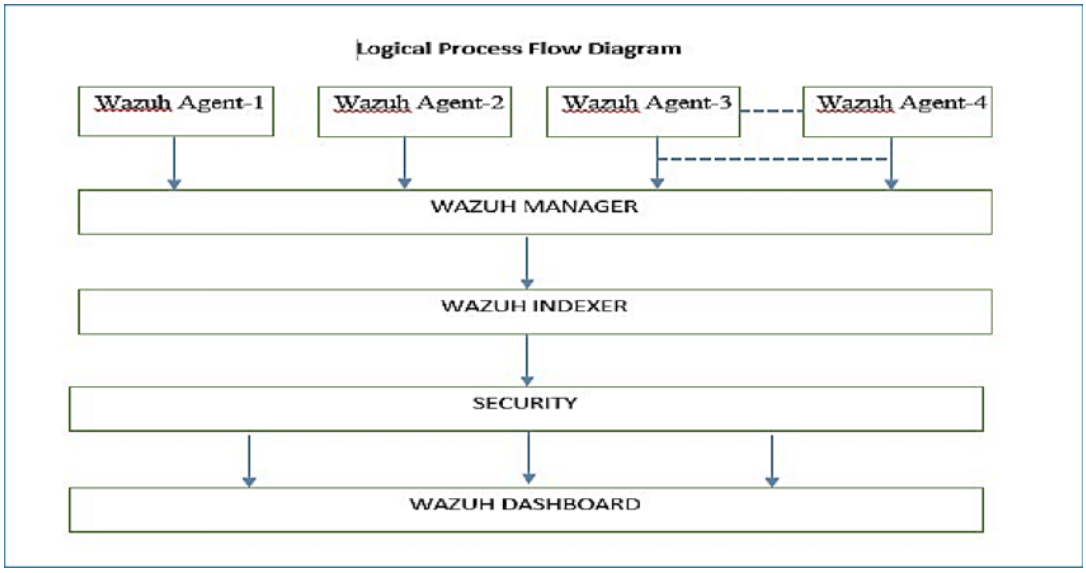


Diagram Showing the Communication Between Wazuh Agent's Installed on Hosts and Blue Team Analysts

Each agent was registered using the secure enrollment key method with group assignment (team16-production) and custom ossec.conf profiles. Agents communicate exclusively on port 1514/UDP (events) and 1515/TCP (enrollment). We enabled full FIM (File Integrity Monitoring) with real-time scanning on all flag locations, configuration directories, and application code folders, vulnerability detection (SCA), and command monitoring for suspicious processes.

To protect the Wazuh server itself, we layered multiple firewall controls. In addition to the Sophos SFOS 25.5.1 perimeter firewall and our point-to-point /30 topology, we configured firewalld on Wazuh-SIEM-01 with a dedicated zone (wazuh-agents). Ports 1514/UDP, 1515/TCP, and 5500/TCP are restricted to accept traffic only from the exact internal IP addresses of the five enrolled hosts (no CIDR, explicit IPs only). Port 443/TCP (HTTPS) is allowed only from authorized subnets for dashboard access, threat hunting. Port 22/TCP (SSH) is completely disabled on all interfaces.



Logical Process Flow Diagram of Wazuh System

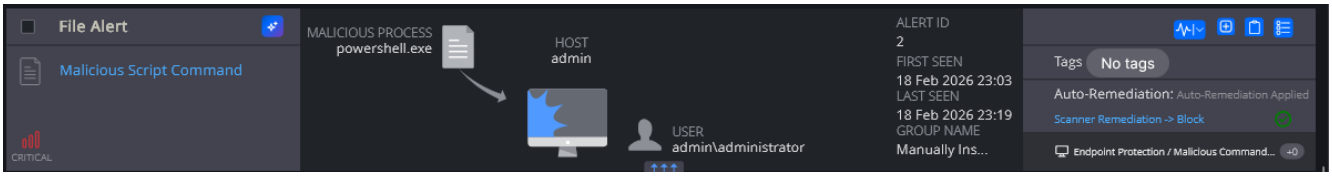
Cynet360 Configuration on Endpoints

Cynet XDR delivered advanced behavioral analysis, automated threat remediation, process tree visibility, MITRE ATT&CK mapping, and one-click response actions, serving as our primary endpoint detection and response (EDR/XDR) layer that complements Wazuh SIEM and the Sophos firewall.

To rigorously validate the deployment and fine-tune response policies, we executed two deliberate test attacks from within the environment:

- **Invoke-Mimikatz** – to simulate credential dumping via PowerShell
- **EICAR test file** – to simulate malware execution

Cynet immediately detected and blocked both tests with high-confidence alerts.
The screenshot below shows the detailed alert generated when 'Invoke-Mimikatz' was executed on the Admin server (hostname: admin, IP: 172.20.10.14, Windows Server 2022). Cynet classified it as **"Malicious Script Command"** (T1059.001 – Command and Scripting Interpreter: PowerShell), automatically applied remediation (Block), and recorded the full process tree from explorer.exe → powershell.exe.



Cynet XDR real-time detection and auto-remediation of Invoke-Mimikatz execution (T1059.001)

The Cynet console confirmed full coverage of the environment. All five hosts appeared as **Active** (or Light Agent) with only minor warnings that were quickly resolved. The endpoint overview and scanned endpoints tables clearly listed every server with its IP, OS, last seen time, and status.

Whenever threats were detected, Cynet automatically triggered its powerful automation engine. During our testing it executed **3 automated response actions**, including Isolate, Disable all NICs, Delete Service, Shut Down, AV Scan, Disable Local User, Remote Shell, and Delete File. The Automation page provided a complete audit trail of every action taken.

Backup Solution for 5 Hosts

We implemented a centralized, SSH-only backup system to meet the requirement of backing up all machines every 30 minutes with enough data for a complete restore. A dedicated AlmaLinux 9 backup server (IP 168.84.5.250) receives pushes from all five scenario machines using rsync over SSH key authentication. This design provides off-machine storage, minimal attack surface, and full coverage of required directories including C:\Users on Windows, /etc /home /root on Linux, application code, and database data. On the Active Directory machine we also perform a full System State backup using wbadmin to ensure complete AD recovery capability.

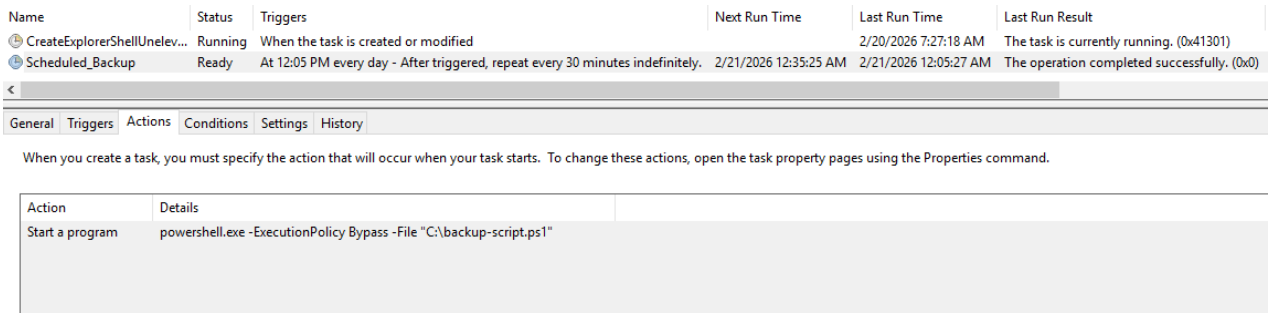
```
[root@localhost backups]# ls -lh
total 0
drwxr-xr-x. 4 root root 38 Feb 17 12:45 ad
drwxr-xr-x. 4 root root 38 Feb 20 11:04 admin
drwxr-xr-x. 6 root root 51 Feb 17 12:45 mgmt
drwxr-xr-x. 2 root root  6 Feb 17 11:22 public
drwxr-xr-x. 3 root root 19 Feb 17 13:58 tickets
drwxr-xr-x. 2 root root  6 Feb 17 11:22 versions
drwxr-xr-x. 5 root root 41 Feb 17 13:00 www
[root@localhost backups]#
```

Central backup server directory showing all machine's Backup Directories

On Windows machines (AD, Ticketing, Admin UI) we installed cwRsync Client in C:\bin and created a PowerShell script (backup-script.ps1) that backs up C:\Users and runs wbadmin for System State on the AD machine. On Linux machines (mgmt and www) we created /root/backup-to-central.sh that backs up /etc, /home, /root and on mgmt also dumps the full MariaDB database plus the data directory. All transfers use rsync with --delete to keep an exact mirror. The scripts ignore locked files and non-critical errors to ensure they complete reliably every run.

Automation was achieved using Windows Task Scheduler on the three Windows machines (highest privileges, repeat every 30 minutes indefinitely) and cron on the two Linux machines (*/30 * * * *). Both log to central log files (C:\backups\central-backup.log and /var/log/central-backup.log) for easy verification. Daily versioning on the backup server keeps 14 days of history in /backups/versions/.

The backup server itself has strict firewall rules allowing SSH only from our five machines and key-only authentication.
This solution fully satisfies the scenario requirements. We tested restores during setup and confirmed all required data (flags, app code, configs, database, and AD System State) can be recovered quickly. The system is simple to monitor and provides real protection against Red Team actions.



Name	Status	Triggers	Next Run Time	Last Run Time	Last Run Result
CreateExplorerShellUnelev...	Running	When the task is created or modified		2/20/2026 7:27:18 AM	The task is currently running. (0x41301)
Scheduled_Backup	Ready	At 12:05 PM every day - After triggered, repeat every 30 minutes indefinitely.	2/21/2026 12:35:25 AM	2/21/2026 12:05:27 AM	The operation completed successfully. (0x0)

General | Triggers | Actions | Conditions | Settings | History

When you create a task, you must specify the action that will occur when your task starts. To change these actions, open the task property pages using the Properties command.

Action	Details
Start a program	powershell.exe -ExecutionPolicy Bypass -File "C:\backup-script.ps1"

Screenshot from Windows Task Scheduler – 30-minute automated backup

Application Hardening

Tickets Server

CHanged the code that it does not return the certificate on the /api/tickets/help endpoint

```c#
using System.Security.Cryptography;
using System.Security.Cryptography.X509Certificates;
using System.Text;
using System.Text.Json;

namespace tickets.Services
{
    public class TicketSigner : ITicketSigner
    {
        private readonly X509Certificate2 _signingCert;
        private readonly ILogger<TicketSigner> _logger;

        public TicketSigner(X509Certificate2 signingCert, ILogger<TicketSigner> logger)
        {
            _signingCert = signingCert ?? throw new
ArgumentNullException(nameof(signingCert));
            _logger = logger;
        }

        public async Task<(string payloadBase64, string signatureBase64, string alg, string
kid)> SignAsync(
            string subject, long validFrom, long validUntil)
        {
            if (string.IsNullOrWhiteSpace(subject))
                throw new ArgumentException("Subject is required", nameof(subject));

            if (validUntil <= validFrom)
                throw new ArgumentException("validUntil must be greater than validFrom");
```

```
        // === SECURITY: Enforce maximum ticket lifetime (4 hours max) ===
        const long MAX_VALIDITY_SECONDS = 4 * 3600; // 4 hours
        if (validUntil - validFrom > MAX_VALIDITY_SECONDS)
            throw new ArgumentException($"Ticket validity cannot exceed
{MAX_VALIDITY_SECONDS} seconds");

        var payload = new Dictionary<string, object>
        {
            ["iss"] = "ticket-signer",
            ["sub"] = subject,
            ["nbf"] = validFrom,
            ["exp"] = validUntil,
            ["iat"] = DateTimeOffset.UtcNow.ToUnixTimeSeconds(),
        };

        // Canonical JSON (sorted keys)
        var ordered = payload.OrderBy(kv => kv.Key, StringComparer.Ordinal);
        var options = new JsonSerializerOptions { WriteIndented = false };

        using var ms = new MemoryStream();
        using (var writer = new Utf8JsonWriter(ms, new JsonWriterOptions { Indented =
false }))
        {
            writer.WriteStartObject();
            foreach (var kv in ordered)
            {
                writer.WritePropertyName(kv.Key);
                JsonSerializer.Serialize(writer, kv.Value, kv.Value?.GetType() ??
typeof(object), options);
            }
            writer.WriteEndObject();
        }

        var payloadBytes = ms.ToArray();
        var payloadBase64 = Convert.ToBase64String(payloadBytes);

        using var rsa = _signingCert.GetRSAPrivateKey();
```

```
            if (rsa == null)
                throw new InvalidOperationException("Certificate does not have RSA private
key");

            var signature = rsa.SignData(payloadBytes, HashAlgorithmName.SHA256,
RSASignaturePadding.Pkcs1);
            var signatureBase64 = Convert.ToBase64String(signature);

            var kid = _signingCert.Thumbprint ?? string.Empty;

            _logger.LogInformation("Ticket signed for subject {Subject} (valid until
{ValidUntil})",
                subject, validUntil);

            return (payloadBase64, signatureBase64, "RS256", kid);
        }

        // === SECURITY: Completely disable or restrict Help() ===
        public Task<string> Help()
        {
            // Return safe message instead of leaking private key
            return Task.FromResult("Ticket signing service - use /sign and /validate
endpoints");
        }
    }
}
```

Admin Server (admin.team16.isucdc.com)

disabled the register feature, because an admin should not be able to self-register in a public-facing server:

![[Pasted image 20260221002220.png]]

using .env file instead of hardcoding credentials:

```
from datetime import timedelta
import os
from dotenv import load_dotenv

load_dotenv()

class Config:
    PERMANENT_SESSION_LIFETIME = timedelta(hours=1)
    SECRET_KEY = os.getenv("SECRET_KEY")
    SESSION_COOKIE_SECURE = os.getenv("SESSION_COOKIE_SECURE",
"False").lower() == "true"
    SESSION_COOKIE_HTTPONLY = True
    SESSION_COOKIE_SAMESITE = "Lax"

    BACKEND_URL = os.getenv("BACKEND_URL")

    # Secure session settings - different name per frontend
    SESSION_COOKIE_NAME = "admin_session"   # ← change to "admin_session" in
admin frontend
    SESSION_COOKIE_HTTPONLY = True
    SESSION_COOKIE_SAMESITE = "Lax"
    SESSION_COOKIE_SECURE = False
```

Removed all passwords, keys, and flags from source code to prevent leaks, and used best practice for cookies such as "HTTPONLY" to disable javascript from accessing the cookies and setting samesite as "LAX".

3.

routes.py

Added a check that checks if the user is member of transit admins group in AD in order to be able to login, the users also have their unique session cookies:

```
...
if response.status_code == 200:
    data = response.json()

    # ←←← SECURITY ENFORCEMENT: Only Transit Admins allowed
    if not data.get("is_admin", False):
        return render_template(
            "login.html",
            error="Access denied: Administrator privileges required"
        ), 403

    # Secure session handling
    session.clear()
    session.permanent = True
    session["username"] = username
    session["is_admin"] = True
    return redirect(url_for("default.index"))
```

4.

routes.py

added the following line

```
session.clear()          # Prevents session fixation
session.permanent = True # Makes session respect PERMANENT_SESSION_LIFETIME
```

Clears any previous session on login to prevent session fixation attacks and ensures proper session lifetime.

WWW Server (www.team16.isucdc.com)

Changed secret key and cookie set to true to not be able to get with XSS:

1.

auth.py

```
if resp.status_code == 200:
    data = resp.json()

    # === SECURITY ENFORCEMENT: Block Admins from citizen frontend ===
    if data.get("is_admin", False):
        return jsonify({"error": "Administrators must use the admin portal"}), 403

    # === SECURE SESSION HANDLING ===
    session.clear()          # ← clears old session
    session.permanent = True # ← makes session last for
PERMANENT_SESSION_LIFETIME

    session["username"] = request.form.get("username")
    return routes.index()
```

- Prevents users with admin privileges from logging into the citizen frontend.
- Adds secure session handling to prevent session fixation attacks.
- Does not allow other groups than Citizens to authenticate

2.

config.py

```
from datetime import timedelta
import os
from dotenv import load_dotenv

load_dotenv()

class Config:
    PERMANENT_SESSION_LIFETIME = timedelta(hours=1)
    SECRET_KEY = os.getenv("SECRET_KEY")
    SESSION_COOKIE_SECURE = os.getenv("SESSION_COOKIE_SECURE",
"False").lower() == "true"
    SESSION_COOKIE_HTTPONLY = True
    SESSION_COOKIE_SAMESITE = "Lax"

    BACKEND_URL = os.getenv("BACKEND_URL")

    # Citizen-specific secure cookie name
    SESSION_COOKIE_NAME = "citizen_session"
    SESSION_COOKIE_HTTPONLY = True
    SESSION_COOKIE_SAMESITE = "Lax"
    SESSION_COOKIE_SECURE = False
```

- Moved all secrets to .env file (no more hardcoded values).
- Used a unique cookie name (citizen_session) to separate it from the admin frontend.
- Added secure cookie flags (HttpOnly, SameSite) for better protection.

MGMT Server (mgmt.team16.isucdc.com)

edited the file that the created users will be specifically added to the citizens group, and removed from the domain users group

/home/cdc/icdc26-backend-mgmt/app/utils/ldap.py
```python
import ssl

from ldap3.extend.microsoft.removeMembersFromGroups import
ad_remove_members_from_groups

from ldap3 import ALL, MODIFY_REPLACE, NTLM, Connection, Server, Tls

from ldap3.core.exceptions import LDAPBindError

from ldap3.extend.microsoft.addMembersToGroups import ad_add_members_to_groups

from ldap3.extend.microsoft.modifyPassword import ad_modify_password



from app.config import Config




def get_ldap_connection(username=Config.LDAP_USER,
password=Config.LDAP_PASS):
```

```
    tls = Tls(validate=ssl.CERT_NONE)

    server = Server(Config.LDAP_SERVER, get_info=ALL, port=636, use_ssl=True,
tls=tls)

    conn = Connection(

        server,

        user=f"{Config.LDAP_DOMAIN}\\{username}",

        password=password,

        authentication=NTLM,

        auto_bind=True,

    )

    return conn




def ldap_exists(username):

    conn = get_ldap_connection()

    conn.search(

        Config.LDAP_DN, f"(sAMAccountName={username})",
attributes=["sAMAccountName"]

    )

    exists = len(conn.entries) > 0
```

```
conn.unbind()

return exists




user_dn = f"cn={username}," + Config.LDAP_DN



user_attributes = {

    "objectClass": ["top", "person", "organizationalPerson", "user"],

    "cn": username,

    "sAMAccountName": username,

    "userPrincipalName": f"{username}@{Config.LDAP_DOMAIN}",

    "displayName": username,

}



if not conn.add(user_dn, attributes=user_attributes):

    conn.unbind()

    return False
```

```
ad_modify_password(conn, user_dn, password, None)


conn.modify(
    user_dn,
    {"userAccountControl": [(MODIFY_REPLACE, [512])]},
)


add_to_group(username, Config.ADMIN_GROUP)


citizens_rid = get_group_rid(Config.ADMIN_GROUP)
if citizens_rid:
    conn.modify(
        user_dn,
        {"primaryGroupID": [(MODIFY_REPLACE, [citizens_rid])]}
    )


    domain_users_dn = f"cn=Domain Users," + Config.LDAP_DN
    ad_remove_members_from_groups(
        conn,
```

```
            user_dn,
            [domain_users_dn],
            fix=True
        )
    else:
        print(f"[LDAP] WARNING: Could not change primary group for {username} - user will remain in Domain Users")


    conn.unbind()
    return True   # changed to bool for clarity (you can keep your original result check if you prefer)


def get_group_rid(group_name):
    """Get the RID of a group via primaryGroupToken (cleanest & most reliable method)."""
    conn = get_ldap_connection()
    conn.search(
        Config.LDAP_DN,
        f"(cn={group_name})",
        attributes=['primaryGroupToken']
```

```
    )
    if conn.entries and hasattr(conn.entries[0], 'primaryGroupToken'):
        rid = int(conn.entries[0].primaryGroupToken.value)
        conn.unbind()
        return rid
    conn.unbind()
    print(f"[LDAP] WARNING: Could not find RID for group '{group_name}'")
    return None




def add_to_group(username, group):
    conn = get_ldap_connection()
    ad_add_members_to_groups(
        conn, f"cn={username}," + Config.LDAP_DN, [f"cn={group}," + Config.LDAP_DN]
    )
    conn.unbind()
    return conn.result["description"] == "success"
```

```
def change_ldap_password(username, password):
    conn = get_ldap_connection()
    ad_modify_password(conn, f"cn={username}," + Config.LDAP_DN, password, None)
    conn.unbind()
    return conn.result["description"] == "success"



def check_group_membership(username, group, conn=None):
    if conn is None:
        conn = get_ldap_connection()
    conn.search(
        Config.LDAP_DN, f"(&(objectClass=group)(cn={group}))", attributes=["member"]
    )


    if conn.entries:
        members = conn.entries[0].member
        is_member = f"cn={username}," + Config.LDAP_DN in [x.lower() for x in members]
```

```
        conn.unbind()
        return is_member

    conn.unbind()
    return False


def authenticate(username, password):
    try:
        conn = get_ldap_connection(username, password)
        is_admin = check_group_membership(username, Config.ADMIN_GROUP, conn)
        conn.unbind()
        return True, is_admin
    except LDAPBindError:
        return False, False
```

Backend `icdc26-backend-mgmt/app/`

File:

config.py

```
import os
from dotenv import load_dotenv

load_dotenv()   # ← Loads secrets from .env

class Config:
    SECRET_KEY = os.getenv("SECRET_KEY")
    LDAP_USER = os.getenv("LDAP_USER")
    LDAP_PASS = os.getenv("LDAP_PASS")
    CITIZEN_GROUP = os.getenv("CITIZEN_GROUP")
    ADMIN_GROUP = os.getenv("ADMIN_GROUP")

    # Secure session settings
    SESSION_COOKIE_NAME = "transit_session"
    SESSION_COOKIE_HTTPONLY = True
    SESSION_COOKIE_SAMESITE = "Lax"
    SESSION_COOKIE_SECURE = False
```

Why we did this: Hardcoded secrets in Python files are a major security risk (visible in git history, code reviews, etc.). Moving them to .env follows industry best practices (12-factor app) and prevents accidental leaks. Also changed the easy guessable secret keys and users value.

2.

utils/ldap.py

```
add_to_group(username, Config.CITIZEN_GROUP)

citizens_rid = get_group_rid(Config.CITIZEN_GROUP)
if citizens_rid:
```

```
    conn.modify(user_dn, {"primaryGroupID": [(MODIFY_REPLACE, [citizens_rid])]})
    ad_remove_members_from_groups(conn, user_dn, [f"cn=Domain
Users,{Config.LDAP_DN}"], fix=True)
```

Why we did this: every new user should only belong to the Citizens group and not be part of Domain Users. This change enforces that requirement.

3.

auth.py

```
success, is_admin = authenticate(username, password)

if not success:
    return jsonify({"error": "Authentication failed"}), 401

# Secure session handling
session.clear()
session["username"] = username
session["is_admin"] = is_admin
session.permanent = True
```

Replaced the previous fake login (ldap_exists + hardcoded is_admin = True) with real LDAP group membership check. This enforces proper role separation.

4.

auth.py

```
session.clear()          # Prevents session fixation
session.permanent = True
```

Clearing the session on login prevents attackers from fixing a session ID before login. Combined with secure cookie settings, it makes session tampering much harder.

5.
utils/decorators.py
```
def require_admin(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if "username" not in session:
            return jsonify({"error": "Authentication required"}), 401
        if not session.get("is_admin", False):
            return jsonify({"error": "Administrator privileges required"}), 403
        return f(*args, **kwargs)
    return decorated_function
```

To protect admin-only routes (e.g. route updates, vehicle updates) and return clear 403 errors when a normal user tries to access them.

6.
vehicles.py

```
@bp.route("/<vehicle_id>", methods=["POST"])
@require_admin          # ← Only admins can update vehicles
def update_vehicle(vehicle_id):
```

Why we did this: Originally anyone could update vehicles. We added admin protection for security, but kept the original update logic.

7.

utils/ldap.py

```
add_to_group(username, Config.CITIZEN_GROUP)

# Make Citizens the primary group
citizens_rid = get_group_rid(Config.CITIZEN_GROUP)
if citizens_rid:
    conn.modify(user_dn, {"primaryGroupID": [(MODIFY_REPLACE, [citizens_rid])]})

    # Remove user from Domain Users group
    domain_users_dn = f"cn=Domain Users," + Config.LDAP_DN
    ad_remove_members_from_groups(
        conn, user_dn, [domain_users_dn], fix=True
    )
```

```
def get_group_rid(group_name):
    # Returns the RID so we can set primaryGroupID
```

Why we did this: every new user must only belong to the Citizens group and must not be part of Domain Users. This is the standard and cleanest way to achieve that in Active Directory.